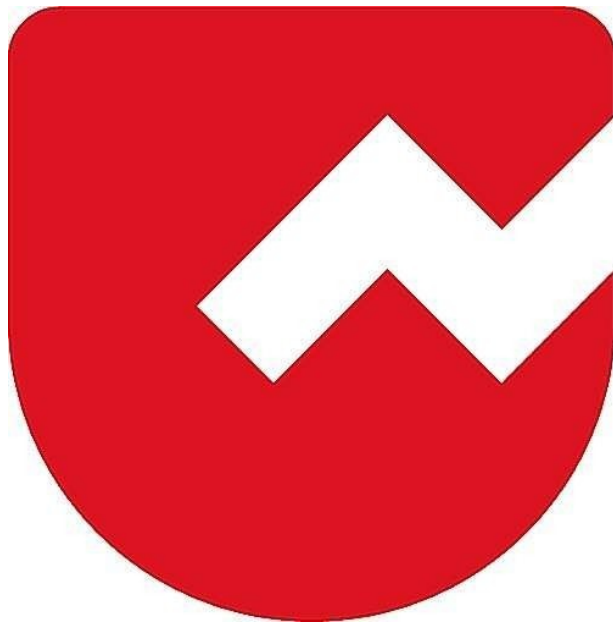




Zoho Schools for Graduate Studies



Notes

1. What are Overloaded Methods?

Methods that share the same name but differ in their parameter list are called **overloaded methods**.

Example:

```
package part1;
public class OverLoadDemo {
public static void main(String[] args)
{
OverLoadDemo o=new OverLoadDemo();
o.printMessage("Happy Teacher's Day!");
o.printMessage("Happy Teacher's Day to Mr. ", " Gnanavel");
}
void printMessage(String msg)
{
System.out.println(msg);
}
void printMessage(String msg1, String msg2)
{
System.out.println(msg1 + msg2);
}
}
```

2. When to prefer overloading (or) Why overloading?

Overloading is preferred whenever a similar type of operation needs to be performed on different kinds of data.

Example: `add(int a, int b)` and `add(double x, double y)`

3. What is the advantage of overloading?

Method overloading offers convenience to the programmer.

Example:

The developer does not need to create and remember unique method names whenever the same operation is performed on different data types.

4. What is the qualifying criteria to determine method overloading?

The following are the two main criteria:

1. Overloaded methods must share the same name.
2. Overloaded methods must differ in their parameter list (number of parameters, type of parameters, or order of parameters).

Rules :

1. Which overloaded method is invoked is decided at compile time based on the type of the arguments.

```
public class OverLoadDemo {
    public static void main(String[] args) {
        OverLoadDemo o = new OverLoadDemo();
        o.add((double)34, (float)56);
        o.add((float)34, (double)56);
    }

    void add(double a, float b) {
        System.out.println("DoubleFloat sum = " + (a + b));
    }

    void add(float a, double b) {
        System.out.println("FloatDouble sum = " + (a + b));
    }
}
```

OUTPUT:

DoubleFloat sum = 90.0

FloatDouble sum = 90.0

2. Overloaded methods must share same name

```
package part1;
```

```
public class OverLoadDemo {
    public static void main(String[] args) {
        OverLoadDemo o = new OverLoadDemo();
        o.add(34.0, 56.4f); // matches add(double, float)
        o.add(34.5f, 57.0); // matches add(float, double)
    }

    void add(double a, float b) {
        System.out.println("DoubleFloat sum = " + (a + b));
    }

    void add(float a, double b) {
        System.out.println("FloatDouble sum = " + (a + b));
    }
}
```

OUTPUT:

DoubleFloat sum = 90.4

FloatDouble sum = 91.5

3. Overloaded methods must vary in parameter list

```
class CorrectOverload {  
    void add(int a, int b) {  
        System.out.println("Sum (int,int) = " + (a + b));  
    }  
    void add(double a, double b) {  
        System.out.println("Sum (double,double) = " + (a + b));  
    }  
    public static void main(String[] args) {  
        CorrectOverload o = new CorrectOverload();  
        o.add(10, 20);  
        o.add(5.5, 6.5);  
    }  
}
```

OUTPUT:

Sum (int,int) = 30

Sum (double,double) = 12.0

4. Overloaded methods can vary in return type

```
package part1;  
public class OverLoadDemo {  
    public static void main(String[] args) {  
        OverLoadDemo o = new OverLoadDemo();  
        int r1 = o.add(34, 56);    // calls add(int,int)  
        double r2 = o.add(34.5, 57); // calls add(double,int)  
        System.out.println("Result from int add = " + r1);  
        System.out.println("Result from double add = " + r2);  
    }  
    int add(int a, int b) {  
        return a + b;  
    }  
    double add(double a, int b) {  
        return a + b;  
    }  
}
```

OUTPUT:

Result from int add = 90

Result from double add = 91.5

5. Overloaded methods can vary in access modifier

```
class OverLoadDemo {  
    public void add(int a, int b) {  
        System.out.println("Public add(int,int) = " + (a + b));  
    }  
    private void add(double a, double b) {  
        System.out.println("Private add(double,double) = " + (a + b));  
    }  
    protected void add(int a, double b) {  
        System.out.println("Protected add(int,double) = " + (a + b));  
    }  
    public static void main(String[] args) {  
        OverLoadDemo o = new OverLoadDemo();  
        o.add(10, 20);  
        o.add(5.5, 6.5);  
        o.add(10, 6.5);  
    }  
}
```

OUTPUT:

Public add(int,int) = 30

Private add(double,double) = 12.0

Protected add(int,double) = 16.5

6. Overloaded methods can throw any new unchecked or checked Exceptions

```
package part1;  
import java.io.FileNotFoundException;  
public class OverLoadDemo {  
    public static void main(String[] args) throws Exception {  
        OverLoadDemo o = new OverLoadDemo();  
        o.add(34, 56.4f); // may throw unchecked exception  
        o.add(34.5f, 57); // no exception  
        o.add(10, 20);    // may throw checked + unchecked exception  
    }  
    static public void add(double a, float b) throws ArithmeticException {  
        System.out.println("DoubleFloat sum = " + (a + b));  
    }  
}
```

```

    }
    void add(float a, double b) {
        System.out.println("FloatDouble sum = " + (a + b));
    }
    static private int add(int a, int b) throws ArithmeticException,
    FileNotFoundException {
        System.out.println("IntInt sum = " + (a + b));
        return (a + b);
    }
}

```

OUTPUT:

```

DoubleFloat sum = 90.4
FloatDouble sum = 91.0
IntInt sum = 30

```

7. Overloaded methods can throw zero or fewer or more exceptions

```

package part1;
import java.io.FileNotFoundException;
public class OverLoadDemo {
    public static void main(String[] args) throws Exception {
        OverLoadDemo o = new OverLoadDemo();

        o.add(10, 20);
        o.add(10.5, 20.5);
        o.add("Hello", "Java");
    }
    void add(int a, int b) {
        System.out.println("Sum (int,int) = " + (a + b));
    }
    void add(double a, double b) throws FileNotFoundException {
        System.out.println("Sum (double,double) = " + (a + b));
    }
    void add(String a, String b) throws FileNotFoundException,
    ArithmeticException {
        System.out.println("Concatenation = " + (a + b));
    }
}

```

OUTPUT:

```

Sum (int,int) = 30
Sum (double,double) = 31.0
Concatenation = HelloJava

```

8. Overloaded methods can appear within the same class or can occur in the sub-class too.

9. Which overloaded method is invoked is only decided at compile time based on the reference type.

10. Method overloading is possible even without inheritance.

11. Static methods can be overloaded

```
package part1;
```

```
public class OverLoadDemo {  
    static void add(int a, int b) {  
        System.out.println("Static add(int,int) = " + (a + b));  
    }  
    static void add(double a, double b) {  
        System.out.println("Static add(double,double) = " + (a + b));  
    }  
    public static void main(String[] args) {  
        OverLoadDemo.add(10, 20);  
        OverLoadDemo.add(10.5, 20.5);  
    }  
}
```

OUTPUT:

```
Static add(int,int) = 30  
Static add(double,double) = 31.0
```

12. Method overloading is possible even without inheritance.

13. There must exist only one valid overloaded method for any call to a overloaded method. Otherwise, it leads to ambiguity when more than one valid overloaded method exist.

```
package part1;  
public class OverLoadDemo {  
    public static void main(String[] args) {  
        OverLoadDemo o=new OverLoadDemo();  
        o.add(34,56.4f);  
        o.add(34.5f,57);  
    }  
    void add(double a, float b)
```

```

{
System.out.println("DoubleFloat sum=" + (a + b));
}
void add(float a, double b)
{
System.out.println("FloatDouble sum=" + (a + b));
}

```

Explanation:

1. The error is **Ambiguity Error during method overloading resolution** (not runtime, but at compile-time).
2. You will get a compiler error: "reference to add is ambiguous". Because the compiler cannot decide which overloaded method to call when both are equally valid after type promotion.

14. 'main' method can be overloaded but cannot be overridden.

```

package part1;
public class OverLoadDemo {
    public static void main(String[] args) {
        System.out.println("Inside main(String[] args)");
        main(10);
        main("Hello Java");
    }
    public static void main(int a) {
        System.out.println("Inside main(int): " + a);
    }
    public static void main(String msg) {
        System.out.println("Inside main(String): " + msg);
    }
}

```

Output:

```

Inside main(String[] args)
Inside main(int): 10
Inside main(String): Hello Java

```

15. constructors can be overloaded but cannot be overridden.

```

package part1;
class Student {
    int id;
}

```



```

String name;
Student() {
    System.out.println("Default constructor called");
}
Student(int id) {
    this.id = id;
    System.out.println("Constructor with id: " + id);
}
}
public class OverLoadDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101);

    }
}

```

OUTPUT:

Default constructor called
 Constructor with id: 101

```

package part1;
public class OverLoadDemo {
    public static void main(String[] args) {
        OverLoadDemo o = new OverLoadDemo();

        o.add(34, 56);    //int,int = exact match
        o.add(34.5, 56); // double,int = matches add(double,int)

        //double cannot be passed to int
        o.add(34.5, 56.7);
    }
    void add(int a, int b) {
        System.out.println("Int version: " + (a + b));
    }
    void add(double a, int b) {
        System.out.println("Double version: " + (a + b));
    }
}

```

OUTPUT:

1. First two calls work fine and print results.
Int version: 90
Double version: 90.5
2. Third call give compile-time error (double cannot be passed to int).

1. Vary in number of parameters

```
void s(int a) { }  
void s(int a, int b) { }
```

Explanation:

- Overloaded methods must differ in parameter list. Here, one method has 1 parameter, another has 2 parameters.
- Here, one method has **1 parameter**, another has **2 parameters**.

2. Vary in order of data types

```
void s(int a, float b) { }  
void s(float a, int b) { }
```

Explanation:

- Overloaded methods can have same number of parameters but different types or order.
- Here, both have 2 parameters, but order is different (int, float vs float, int).

3. Same parameter types (Not allowed)

```
void s(int a) { }  
void s(int b) { }
```

Explanation:

- Overloaded methods cannot have the same parameter types and same order.
- Only return type or variable name is different here, which does not count.
- This will give compile-time error: *“method s(int) is already defined”*.